

```
""" Ohjelma etsii käyttäjän haluamista osakkeista käyttäjän haluamat tiedot,
käyttäen yfinance kirjastoa tiedonhakuun, sekä pandas kirjastoa tiedon
tallentamiseen. Tiedonhaun nopeuttamiseen käytetään concurrent.futures
kirjastosta saatavia workereita: 'ThreadPoolExecutor'. API-kutsujen yli-
kuormitus vältetään pitämällä 300 osakkeen välein 3 minuutin (180 sekunnin)
tauko. Osakkeet tallennetaan käyttäjän nimeämään tiedostoon.
"""
```

```
# kirjastot
import yfinance as yf
import pandas as pd
import time
from concurrent.futures import ThreadPoolExecutor
```

```
def sorts_stocks():
    """ Tallentaa osakkeiden symbolit tekstitiedostosta listaksi

    :return: palauttaa listan osakkeista
    """
```

```
    save_place = []
    filename = "stocks_to_search"
    try:
        with open(filename, "r") as file:
            for line in file:
                save_place.append(line.strip().upper())
    except FileNotFoundError:
        print("File not found. Program will exit shortly.")
        quit()
    return save_place
```

```
def gather_data(symbol, criteria):
    """ Gathers data from yfinance

    """

    try:
        print(f"Haetaan: {symbol}...")
        stock = yf.Ticker(symbol)
        info = stock.info

        data = {"Ticker": symbol}
        for limit in criteria:
            data[limit] = info.get(limit)

        # Haetaan historia
        history = stock.history(period="1y")
        if not history.empty:
            end_price = history['Close']
```

```
# laskee osakkeen prosentuaalisen kokonaistuoton
data['Yearly_Return'] = (end_price.iloc[-1] / end_price.iloc[0]) - 1

# laskee hinta vaihtelun
data['Yearly_Volatility'] = end_price.pct_change().std() * (252 **
0.5)

    return data
except Exception as e:
    print(f"Virhe tickerillä {symbol}: {e}")
    return None

def workers(stocks, criteria):
    results = []
    num_workers = 5 # saa muuttaa mutta max 10
    chunk_size = 300 # Käsitellään 300 osaketta kerrallaan
    pause_duration = 180 # 3 minuuttia

    # Jaetaan osakelista 300 osakkeen paloihin
    for i in range(0, len(stocks), chunk_size):
        chunk = stocks[i: i + chunk_size]

        print(f"\n--- Aloitetaan erä: osakkeet {i + 1} - {min(i + chunk_size,
len(stocks))} ---")

        with ThreadPoolExecutor(max_workers=num_workers) as executor:
            future_to_stock = {executor.submit(gather_data, s, criteria): s for s
in chunk}

            for future in future_to_stock:
                res = future.result()
                if res:
                    results.append(res)

    # Jos listassa on vielä osakkeita jäljellä tämän erän jälkeen, pidetään
    tauko

    if i + chunk_size < len(stocks):
        print(f"\n300 osaketta haettu. Pidetään 3 minuutin (300s) tauko...")
        time.sleep(pause_duration)
        print("Tauko ohi, jatketaan hakuja.\n")

    return results

def saving_data(everything, file):
    """ tallentaa kerätyn datan csv tiedostoon

    """

    if not everything:
        print("Ei dataa tallennettavaksi.")
        return
    df = pd.DataFrame(everything)
```

```
df.to_csv(file, index=False, sep=';', encoding='utf-8-sig')
print(f"Data tallennettu onnistuneesti: {file}")

def main():

    # kriteerit (muokkaa kutakin hakua varten(Yearly_return, Yearly_Volatility
    # haetaan aina laskuja varten))

    # vaihda kriteerit aina
    criteria = ['longName', 'sector', 'industry', 'marketCap', 'trailingPE',
                'priceToBook', 'beta', 'returnOnEquity', 'revenueGrowth',
'debtToEquity',
                'operatingMargins', 'dividendYield']

    # tallentaa osakkeet
    stocks = sorts_stocks()

    print(f"Löydetty {len(stocks)} symbolia tiedostosta.")
    where = input("Anna tallennettavan .csv-tiedoston nimi (esim. tulokset.csv):
")

    print("Kerätään dataa lähteistä...")
    all_data = workers(stocks, criteria)

    saving_data(all_data, where)

if __name__ == "__main__":
    main()
```